

BleX: Multi-Tenant Architecture and Bug Fix Proposal

Technical Proposal | May 2026

Proposed by Akshat | sigmatic.ai

Background

BleX is a BLE asset tracking platform for industrial and commercial deployments. An Android tablet acts as a local hub: it runs an embedded MQTT broker, scans BLE beacons directly, and bridges data to a DGX server. A Python master engine on the DGX server applies zone logic (Kalman filter, hysteresis, dwell-time) and stores zone-change events via a FastAPI backend in PostgreSQL. The platform is currently single-tenant. Three demo deployments are in progress and the system needs to support 20 to 200 tenants.



Current data flow (single-tenant)

Current Issues

Issue 1 - Scanner Setup Bug. When an IT person pushes wrong WiFi credentials to a Pi scanner via the Android app, the Pi drops off the setup network and gets stuck with no connectivity. Recovery requires manually power cycling the device.

Issue 2 - No Tenant Isolation. All scanners publish to the same MQTT topic namespace. Two customers' data collide on a shared broker. There is no tenant concept in the database, API, or master engine.

Fix 1: Scanner WiFi Provisioning Recovery

How setup works

- Pi boots and connects to the tablet hotspot (setup WiFi, credentials baked into the Pi image).
- Pi appears in the BleX app under Configurator then Scanners via UDP discovery on the shared LAN.
- IT person selects the Pi and enters site WiFi credentials (hospital or office network).
- App sends credentials to the Pi over HTTP on port 8888.
- Pi attempts to connect to the new network.



Watchdog confirms connection before dropping the setup fallback

The bug

The current code tears down the setup network connection before verifying the new one works. It fires the connect command without waiting for a result. If the new network is unreachable, the Pi has nowhere to fall back to.

The fix

- Keep the setup network connection active until the new WiFi connection is confirmed working.
- Start a background watchdog after pushing credentials: poll for connectivity every 5 seconds for up to 60 seconds.
- On success: confirm connected to the app, then optionally tear down the setup network.
- On timeout: delete the bad WiFi profile, reconnect to the setup network, report failure to the app.

Status endpoint

A new GET /status endpoint on the Pi returns the current state (idle, connecting, connected, or failed). The Android app polls this every 3 seconds and shows live feedback: a spinner while connecting, a success message on completion, or a retry prompt on failure.

Affected files. `current/scanner/provisioner_service.py` | `provisioner/esp32/provision_listener.py` | `Android HotspotTab.kt`

Fix 2: Multi-Tenant Architecture

MQTT Topic Namespacing

- Current format: ble/scanner/{mac}
- New format: ble/{tenant_id}/scanner/{mac}
- Example: ble/dave_house/scanner/AA:BB:CC:DD:EE:FF

The Pi provisioner writes the tenant_id to local config when the IT person sets up the scanner. The Android app prefixes its own BLE scan publications with the same tenant ID. Old Pi images without a tenant_id fall back to the original topic format.

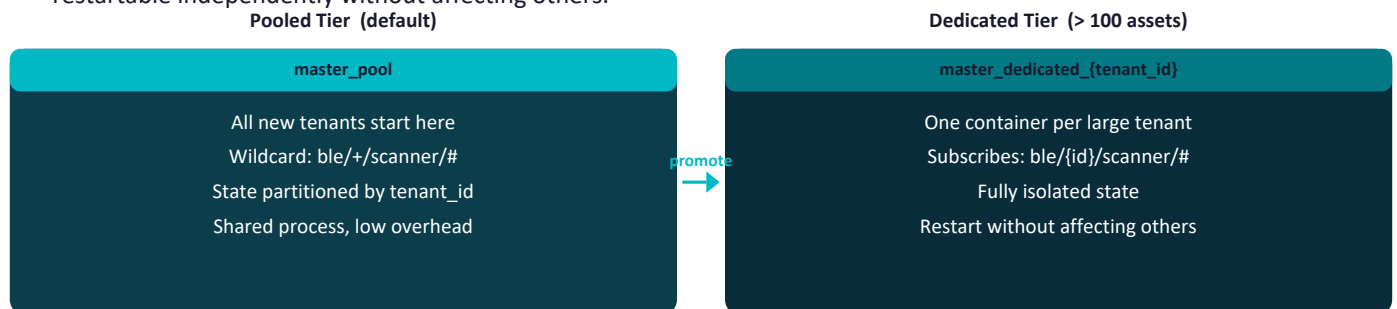
Database: Schema Per Tenant

Each tenant gets a dedicated PostgreSQL schema (e.g. tenant_dave_house) with identical table structures. A global shared schema holds the tenant registry. The FastAPI API routes each request to the correct schema via an X-Tenant-ID header using SQLAlchemy's schema_translate_map. Existing data migrates to a tenant_default schema. A missed query filter cannot expose one customer's data to another because schemas are physically separate.

Master Engine: Tiered Containers

Zone decision logic runs in two container types based on tenant size:

- Pool container. One shared container subscribes to all tenants via a wildcard MQTT topic. In-memory state is partitioned by tenant ID. All new tenants start here.
- Dedicated container. One container per large tenant, subscribing only to that tenant's topic. Fully isolated state, restartable independently without affecting others.



Cron job checks asset count every 5 min. Demotes back to pool if count drops below 20.

A cron job runs every 5 minutes and checks asset counts per tenant. Tenants over 100 registered assets are promoted to a dedicated container automatically. Tenants that drop below 20 return to the pool. Zone state rebuilds from live MQTT within 30 seconds of startup: no state handoff logic needed.

Implementation Phases

Phase	Tasks	Files / Components
1 - Bug Fix (Immediate)	Fix Pi provisioner watchdog. Fix ESP32 provisioner. Update Android Hotspot tab with live status polling.	provisioner_service.py HotspotTab.kt
2 - Tenant Identity	Add tenant_id to Pi MQTT topics. Prefix Android scanner publications. Push tenant config from provisioner to app.	scanner.py MqttManager.kt SettingsManager.kt
3 - DGX Backend	PostgreSQL schema migration. FastAPI tenant routing. Master pool container. Dedicated container mode. FIFO consumer multi-tenant. Tier check cron job.	asset_api routers master_pool.py check_tiers.py docker-compose.yml
4 - Cleanup	Remove master zone logic from Pi image. Archive dead code directory.	Pi image build tags_tracking archive

Initial Tenants

Tenant ID	Owner	Starting Tier
dave_house	Dave	Pool
ehab_house	Ehab	Pool
raghu_home	Raghu	Pool

Open Items

- MQTT ACL per tenant: currently a shared password file. Per-tenant credentials to be added in a later phase.
- Promotion thresholds: 100 assets to promote, 20 to demote. To be tuned after first real deployments.
- FIFO consumer: single consumer with multi-queue polling for now, scale independently if load requires it.